

FreeRTOS信号量 基于STM32



本文详细介绍了FreeRTOS中信号量的基本概念及应用，包括二值信号量、计数信号量、互斥信号量和递归信号量的创建、删除、发送和接收方法，并通过实例演示了信号量在任务同步中的作用。

目录

[概述](#)

[一、信号量基本概念](#)

[1.二值信号量](#)

[2.计数信号量](#)

[3.互斥信号量](#)

[4.递归信号量](#)

[二、二值信号量运作机制](#)

[三、计数信号量运作机制](#)

[四、常用信号量函数接口讲解](#)

[1.创建二值信号量 xSemaphoreCreateBinary\(\)](#)

[2.创建计数信号量 xSemaphoreCreateCounting\(\)](#)

[3.信号量删除函数 vSemaphoreDelete\(\)](#)

[4.信号量释放函数](#)

[4.1 xSemaphoreGive\(\)_\(任务\)](#)

[4.2 xSemaphoreGiveFromISR\(\)_\(中断\)](#)

[五、信号量获取函数](#)

[5.1 xSemaphoreTake\(\)_\(任务\)](#)

[5.2 xSemaphoreTakeFromISR\(\)_\(中断\)](#)

[六、二值信号量同步实验](#)

[七、计数信号量实验](#)

[八、实验现象](#)

概述

同志们，回想一下，你是否在裸机编程中这样使用过一个变量：用于标记某个事件是否发生，或者标志一下某个东西是否正在被使用，如果是被占用了的或者没发生，我们就不对它进行操作。比如说：

停车厂车停车位满了，有一辆车开走了，另一辆车才能停进去。

这个停车位我占了，你们只能等着

在这种情况下我们可以使用[信号量](#)(semaphore)，它更节省内存。

本章涉及如下内容：

- 什么是计数型信号量？什么是二进制信号量？以及互斥信号量和递归信号量
- 怎么创建、删除信号量
- 怎么发送、获得信号量

信号量常常用于控制对共享资源的访问和任务同步。举一个很常见的例子，某个停车场有 100 个停车位，这 100 个停车位大家都可以用，对于大家来说这 100 个停车位就是共享资源。假设现在这个停车场正常运行，你要把车停到这个这个停车场肯定要先看一下现在停了多少车了？还有没有停车位？当前停车数量就是一个信号量，具体的停车数量就是这个信号量值，当这个值到 100 的时候说明停车场满了。停车场满的时你可以等一会看看有没有其他的车开出停车场，当有车开出停车场的时候停车数量就会减一，也就是说信号量减一，此时你就可以把车停进去了，你把车停进去以后停车数量就

会加一，也就是信号量加一。这就是一个典型的使用 信号量进行共享资源管理的案例，在这个案例中使用的就是计数型信号量。再看另外一个案例：使用公共电话，我们知道一次只能一个人使用电话，这个时候公共电话就只可能有两个状态： 使用或未使用，如果用电话的这两个状态作为信号量的话，那么这个就是二值信号量。信号量用于控制共享资源访问的场景相当于一个上锁机制，代码只有获得了这个锁的钥匙 才能够执行。上面我们讲了信号量在共享资源访问中的使用，信号量的另一个重要的应用场合就是任务 同步，用于任务与任务或中断与任务之间的同步。在执行中断服务函数的时候可以通过向任务 发送信号量来通知任务它所期待的事件发生了，当退出中断服务函数以后在任务调度器的调度 下同步的任务就会执行。在编写中断服务函数的时候我们都知道一定要快进快出，中断服务函数里面不能放太多的代码，否则的话会影响的中断的实时性。裸机编写中断服务函数的时候一般都只是在中断服务函数中打个标记，然后在其他的地方根据标记来做具体的处理过程。在使用 RTOS 系统的时候我们就可以借助信号量完成此功能，当中断发生的时候就释放信号量，中 断服务函数不做具体的处理。具体的处理过程做成一个任务，这个任务会获取信号量，如果获 取到信号量就说明中断发生了，那么就开始完成相应的处理，这样做的好处就是中断执行时间 非常短。这个例子就是中断与任务之间使用信号量来完成同步，当然了，任务与任务之间也可以 使用信号量来完成同步。FreeRTOS 中还有一些其他特殊类型的信号量，比如互斥信号量和递归互斥信号量，这些具 体遇到的时候在讲解。有关信号量的知识在 FreeRTOS 的官网上都有详细的讲解，包括二值信 号量、计数型信号量、互斥信号量和递归互斥信号量，

1.二值信号量

二值信号量既可以用于临界资源访问也可以用于同步功能。二值信号量和互斥信号量（以下使用互斥量表示互斥信号量）非常相似，但是有一些细 微差别：互斥量有优先级继承机制，二值信号量则没有这个机制。这使得二值信号量更偏 向应用于同步功能（任务与任务间的同步或任务和中断间同 步），而互斥量更偏向应用于 临界资源的访问。用作同步时，信号量在创建后应被置为空，任务 1 获取信号量而进入阻塞，任务 2 在 某种条件发生后，释放信号量，于是任务 1 获得信号量得以进入就绪态，如果任务 1 的优 先级是最高的，那么就会立即切换任务，从而达到了两个任务间的同步。同样的，在中断 服务函数中释放信号量，任务 1 也会得到信号量，从而达到任务与中断间的同步。还记得我们经常说的中断要快进快出吗，在裸机开发中我们经常是在中断中做一个标 记，然后在退出的时候进行轮询处理，这个就是类似我们使用信号量进行同步的，当标记 发生了，我们再做其他事情。在 FreeRTOS 中我们用信号量用于同步，任务与任务的同步， 中断与任务的同步，可以大大提高效率。可以将二值信号量看作只有一个消息的队列，因此这个队列只能为空或满（因此称为二 值），我们在运用的时候只需要知道队列中是否有消息即可，而无需关注消息是什么。

2.计数信号量

二进制信号量可以被认为是长度为 1 的队列，而计数信号量则可以被认为长度大于 1 的队列，信号量使用者依然不必关心存储在队列中的消息，只需关 心队列是否有消息即可。顾名思义，计数信号量肯定是用于计数的，在实际的使用中，我们常将计数信号量用于事件计数与资源管理。每当某个事件发生时，任务或者中断将释放一个信号量（信号量 计数值加 1），当处理被事件时（一般在任务中处理），处理任务会取走该信号量（信号 量计数值减 1），信号量的计数值则表示还有多少个事件没被处理。此外，系统还有很多 资源，我们也可以使用计数信号量进行资源管理，信号量的计数值表示系统 中可用的资源 数目，任务必须先获取到信号量才能获取资源访问权，当信号量的计数值为零时表示系统 没有可用的资源，但是要注意，在使用完资源的时候必须归还信号量，否则当计数值为 0 的时候任务就无法访问该资源了。计数型信号量允许多个任务对其进行操作，但限制了任务的数量。比如有 一个停车场，里面只有 100 个车位，那么能停的车只有 100 辆，也相当于我们的信号量有 100 个，假如一开始停车场的车位还有 100 个，那么每进去 一辆车就要消耗一个停车位，车位的数量就 要减一，对应的，我们的信号量在使用之后也需要减一，当停车场停满了 100 辆车的时候， 此时的停车位为 0，再来的车就不能停进去了，否则将造成事故，也相当于我们的信号量为 0，后面的任务对这个停车场资源的访问也无法进行，当有车从停车场离 开的时候，车 位又空余出来了，那么，后面的车就能停进去了，我们信号量的操作也是一样的，当我们 释放了这个资源，后面的任务才能对这个资源 进行访问。

3.互斥信号量

互斥信号量其实是特殊的二值信号量，由于其特有的优先级继承机制从而使它更适用 于简单互锁，也就是保护临界资源（什么是优先级继承在后续相信 讲解）。用作互斥时，信号量创建后可用信号量个数应该是满的，任务在需要使用临界资源时，（临界资源是指任何时刻只能被一个任务访问的资 源），先获取互斥信号量，使其变空，这样其他任务需要使用临界资源时就会因为无法获取信号量而进入阻塞，从而保证了临界 资源的安全。在操作系 统中，我们使用信号量的很多时候是为了给临界资源建立一个标志，信号量 表示了该临界资源被占用情况。这样，当一个任务在访问临界资源的时候， 就会先对这个 资源信息进行查询，从而在了解资源被占用的情况之后，再做处理，从而使得临界资源得 到有效的保护。

4.递归信号量

递归信号量，见文知义，递归嘛，就是可以重复获取调用的，本来按照信号量的特性， 每获取一次可用信号量个数就会减少一个，但是递归则不然，对于已经获取递归互斥量的任务可以重复获取该递归互斥量，该任务拥有递归信号量的所有权。任务成功获取几次递 归互斥量，就要返还几次，在此之前 递归互斥量都处于无效状态，其他任务无法获取，只 有持有递归信号量的任务才能获取与释放。

二、二值信号量运作机制

创建信号量时，系统会为创建的信号量对象分配内存，并把可用信号量初始化为用户 自定义的个数，二值信号量的最大可用信号量个数为 1。二值信号 量获取，任何任务都可以从创建的二值信号量资源中获取一个二值信号量， 获取成功则返回正确，否则任务会根据用户指定的阻塞超时时间来等待其它 任务/中断释放 信号量。在等待这段时间，系统将任务变成阻塞态，任务将被挂到该信号量的阻塞等待列 表中。在二值信号量无效的时候，假如此时有 任务获取该信号量的话，那么任务将进入阻塞 状态，具体见图1

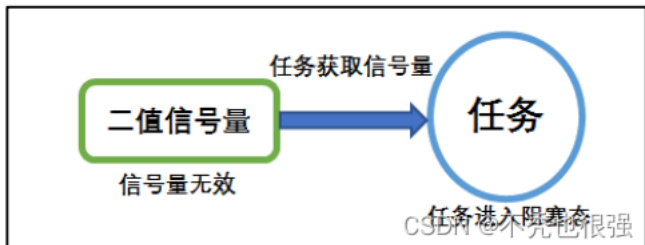


图1 信号量无效时候获取

假如某个时间中断/任务释放了信号量，其过程具体见图 2，那么，由于获取无效信 号量而进入阻塞态的任务将获得信号量并且恢复为就绪态，其过程具体见图3。

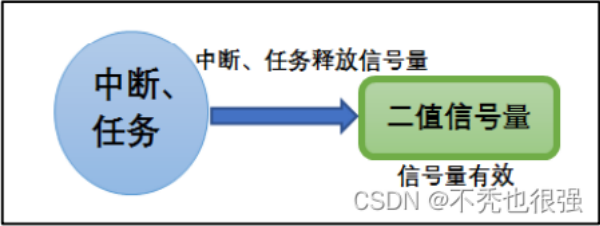


图2 中断、任务释放信号量

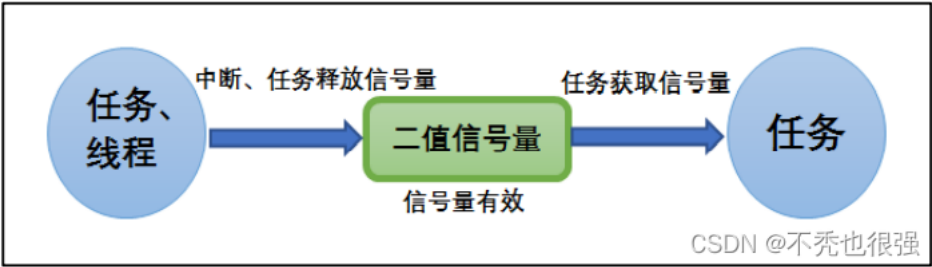


图3 二值信号量运作机制

三、计数信号量运作机制

计数信号量可以用于资源管理，允许多个任务获取信号量访问共享资源，但会限制任 务的最大数目。访问的任务数达到可支持的最大数目时，会阻塞其 他试图获取该信号量的 任务，直到有任务释放了信号量。这就是计数型信号量的运作机制，虽然计数信号量允许多个任务访问同一个资源，但是也有限 定，比如某个资源限定只能有 3 个任务访问，那么 第 4 个任务访问的时候，会因为获取不到信号量而进入阻塞，等到有任务（比如任务 1） 释放掉该资源的时候，第 4 个任务才能获取到信号量从而进行资源的访问，其运作的机制 具体见图4

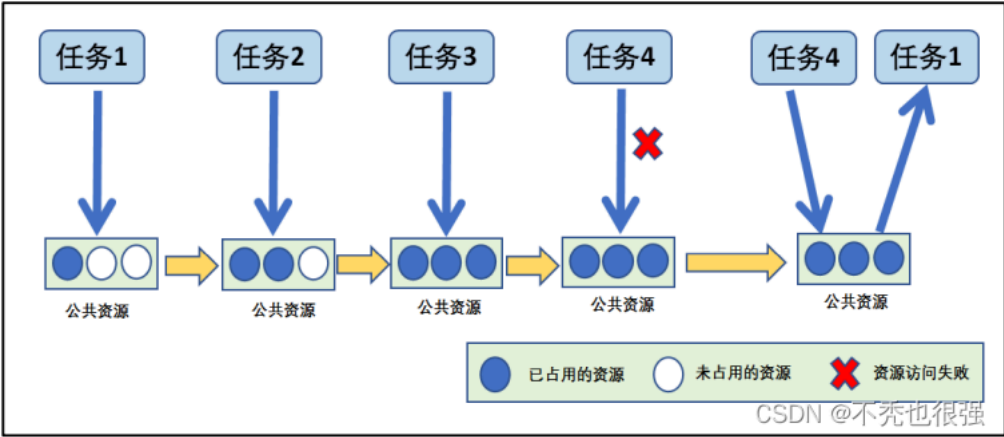


图4 计数信号量运作示意图

四、常用信号量函数接口讲解

1.创建二值信号量 xSemaphoreCreateBinary()

xSemaphoreCreateBinary()用于创建一个二值信号量，并返回一个句柄。其实二值信号 量和互斥量都共同使用一个类型 SemaphoreHandle_t 的句 柄，该句柄的原型是一个 void 型 的 指 针。使用 该 函数 创建 的 二 值 信 号 量 是 空 的 ， 在 使 用 函 数 xSemaphoreTake()获取之前必须先调用函数 xSemaphoreGive()释放后才可以获取。

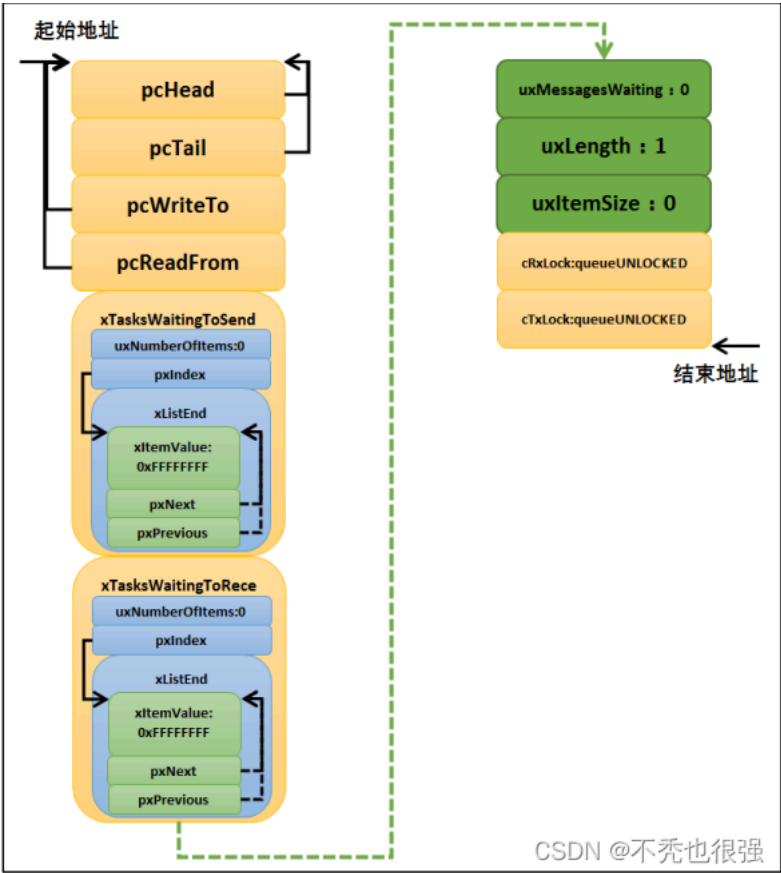


图5 二值信号量创建完成示意图

2. 创建计数信号量 xSemaphoreCreateCounting()

xSemaphoreCreateCounting ()用于创建一个计数信号量。要想使用该函数必须在 FreeRTOSConfig.h 中把宏 configSUPPORT_DYNAMIC_ALLOCATION 定义为 1，即开启动态内存分配。其实该宏在 FreeRTOS.h 中默认为 1，即所有 FreeRTOS 的对象在创建的时候都默认使用动态内存分配方案。其实计数信号量跟二值信号量的创建过程都差不多，其实也是间接调用 xQueueGenericCreate()函数进行创建。

xSemaphoreCreateCounting()函数说明

函数原型	SemaphoreHandle_t xSemaphoreCreateCounting(UBaseType_t uxMaxCount, UBaseType_t uxInitialCount);	
功能	创建一个计数信号量。	
参数	uxMaxCount	计数信号量的最大值，当达到这个值的时候，信号量不能再被释放。
	uxInitialCount	创建计数信号量的初始值。
返回值	如果创建成功则返回一个计数信号量句柄，用于访问创建的计数信号量。如果创建不成功则返回 NULL。	

如果我们创建一个最大计数值为 5，并且默认有效的可用信号量个数为 5 的计数信号量，那么计数信号量创建成功的示意图具体见图6

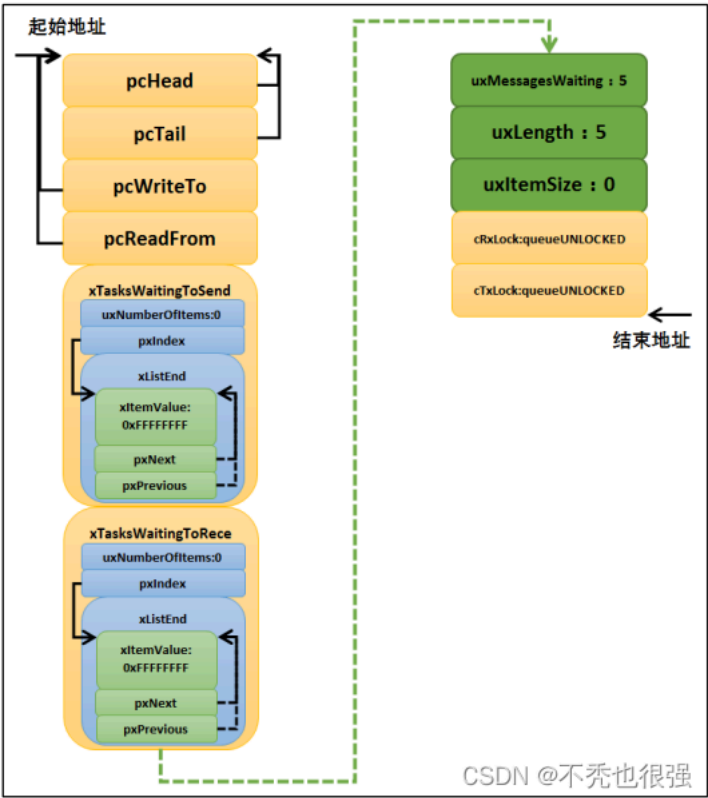


图6 计数信号量创建成功示意图

3.信号量删除函数 vSemaphoreDelete()

vSemaphoreDelete()用于删除一个信号量，包括二值信号量，计数信号量，互斥量和递归互斥量。如果有任务阻塞在该信号量上，那么不要删除该信号量。无返回值

函数原型	void vSemaphoreDelete(SemaphoreHandle_t xSemaphore);	
功能	删除一个信号量。	
参数	xSemaphore	信号量句柄。

删除信号量过程其实就是删除消息队列过程，因为信号量其实就是消息队列，只不过 是无法存储消息的队列而已。

4.信号量释放函数

与消息队列的操作一样，信号量的释放可以在任务、中断中使用，所以需要有不 一样的 API 函数在不 一样的上下文环境中调用。在前面的讲解中，我们知道，当信号量有效的时候，任务才能获取信号量，那么，是 什么函数使得信号量变得有效？其实有两个方式，一个是在创建的时候进行初始化，将它 可用的信号量个数设置一个初始值；在二值信号量中，该初始值的范围是 0~1（旧版本的 FreeRTOS 中创建二值信号量默认是有效的，而新版本则默认是无效），假如初始值为 1 个 可用的信号量的话，被申请一次就变得无效了，那就需要我们释放信号量，FreeRTOS 提供 了信号量释放函数，每调用一次该函数就释放一个信号量。但是有个问题，能不能一直释 放？很显然，这是不能的，无论是你的信号量是二值信号量还是计数信号量，都 要注意可用信号量的范围，当用作二值信号量的时候，必须确保其可用值在 0~1 范围内；而用作计 数信号量的话，其范围是由用户在创建时指定 uxMaxCount，其最大可用信号量不允许超出 uxMaxCount，这代表我们不能一直调用信号量释放函数来释放信号量，其实一直调用也是 无法释放成功的，在写代码的时候，我们要注意代码的严谨性罢了。

4.1 xSemaphoreGive()（任务）

xSemaphoreGive()是一个用于释放信号量的宏，真正的实现过程是调用消息队列通用 发送函数，释放的信号量对象必须是已 经被创建的，可以用于二值信号量、计数信号量、互斥量的释放，但不能释放由函数 xSemaphoreCreateRecursiveMutex()创建的递归互斥量。此外该函数不能在中断中使用。

xSemaphoreGive()函数使用实例

```
1. static void Send_Task(void* parameter)
2. {
3.     BaseType_t xReturn = pdPASS;/* 定义一个创建信息返回值，默认为pdPASS */
4.     while (1)
5.     {
6.         /* K1 被按下 */
```

```

7.  if( Key_Scan(KEY1_GPIO_PORT,KEY1_GPIO_PIN) == KEY_ON )
8.  {
9.      xReturn = xSemaphoreGive( BinarySem_Handle );//给出二值信号量 xSemaphoreGive 释放信号量
10.     if( xReturn == pdTRUE )
11.         printf("BinarySem_Handle二值信号量释放成功!\r\n");
12.     else
13.         printf("BinarySem_Handle二值信号量释放失败!\r\n");
14. }
15. /* K2 被按下 */
16. if( Key_Scan(KEY2_GPIO_PORT,KEY2_GPIO_PIN) == KEY_ON )
17. {
18.     xReturn = xSemaphoreGive( BinarySem_Handle );//给出二值信号量
19.     if( xReturn == pdTRUE )
20.         printf("BinarySem_Handle二值信号量释放成功!\r\n");
21.     else
22.         printf("BinarySem_Handle二值信号量释放失败!\r\n");
23. }
24. vTaskDelay(20);
25. }
26. }

```



4.2 xSemaphoreGiveFromISR()（中断）

用于释放一个信号量，带中断保护。被释放的信号量可以是二进制信号量和计数信号量。和普通版本的释放信号量 API 函数有些许不同，它不能释放互斥量，这是因为互斥量不可以在中断中使用，互斥量的优先级继承机制只能在任务中起作用，而在中断中毫无意义。

xSemaphoreGiveFromISR()函数使用实例

```

1. void vTestISR( void )
2. {
3.     BaseType_t pxHigherPriorityTaskWoken;
4.     uint32_t ulReturn;
5.     /* 进入临界段，临界段可以嵌套 */
6.     ulReturn = taskENTER_CRITICAL_FROM_ISR();
7.
8.     /* 判断是否产生中断 */
9.     {
10.        /* 如果产生中断，清除中断标志位 */
11.
12.        //释放二值信号量，发送接收到新数据标志，供前台程序查询
13.        xSemaphoreGiveFromISR(BinarySem_Handle,&
14.        pxHigherPriorityTaskWoken);
15.
16.        //如果需要的话进行一次任务切换，系统会判断是否需要切换
17.        portYIELD_FROM_ISR(pxHigherPriorityTaskWoken);
18.    }
19.
20.    /* 退出临界段 */
21.    taskEXIT_CRITICAL_FROM_ISR( ulReturn );
22. }

```



五、信号量获取函数

与消息队列的操作一样，信号量的获取可以在任务、中断（中断中使用并不常见）中使用，所以需要有不同的 API 函数在不同的上下文环境中调用。与释放信号量对应的是获取信号量，我们知道，当信号量有效的时候，任务才能获取信号量，当任务获取了某个信号量的时候，该信号量的可用个数就减一，当它减到 0 的时候，任务就无法再获取了，并且获取的任务会进入阻塞态（假如用户指定了阻塞超时时间的话）。如果某个信号量中当前拥有 1 个可用的信号量的话，被获取一次就变得无效了，那么此时另外一个任务获取该信号量的时候，就会无法获取成功，该任务便会进入阻塞态，阻塞时间由用户指定。

5.1xSemaphoreTake()（任务）

xSemaphoreTake()函数用于获取信号量，不带中断保护。获取的信号量对象可以是二值信号量、计数信号量和互斥量，但是递归互斥量并不能使用这个 API 函数获取。其实获取信号量是一个宏，真正调用的函数是 xQueueGenericReceive ()。该宏不能在中断使用，而是必须由具体中断保护功能的 xQueueReceiveFromISR()版本代替。

函数	<pre>#define xSemaphoreTake(xSemaphore, xBlockTime) xQueueGenericReceive((QueueHandle_t) (xSemaphore),</pre>
----	---

👉SDN @不秃也很强

原型	NULL, (xBlockTime), pdFALSE)	
功能	获取一个信号量，可以是二值信号量、计数信号量、互斥量。	
参数	xSemaphore	信号量句柄。
	xBlockTime	等待信号量可用的最大超时时间，单位为 tick（即系统节拍周期）。如果宏 INCLUDE_vTaskSuspend 定义为 1 且形参 xTicksToWait 设置为 portMAX_DELAY，则任务将一直阻塞在该信号量上（即没有超时时间）。
返回值	获取成功则返回 pdTRUE，在指定的超时时间中没有获取成功则返回 errQUEUE_EMPTY。 CSDN @不秃也很强	

从该宏定义可以看出释放信号量实际上是一次消息出队操作，阻塞时间由用户指定 xBlockTime，当有任务试图获取信号量的时候，当且仅当信号量有效的时候，任务才能读 获取到信号量。如果信号量无效，在用户指定的阻塞超时时间中，该任务将保持阻塞状态 以等待信号量有效。当其它任务或中断释放了有效的信号量，该任务将自动由阻塞态转移 为就绪态。当任务等待的时间超过了指定的阻塞时间，即使信号量中还是没有可用信号量，任务也会自动从阻塞态转移为就绪态。

xSemaphoreTake()函数使用实例

```

1. static void Receive_Task(void* parameter)
2. {
3.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
4.     while (1)
5.     {
6.         //获取二值信号量 xSemaphore, 没获取到则一直等待
7.         xReturn = xSemaphoreTake(BinarySem_Handle, /* 二值信号量句柄    xSemaphoreTake获取一个信号量，可以是二值信号量、计数信号量、互斥量。*
8.                                portMAX_DELAY); /* 等待时间 */
9.         if (pdTRUE == xReturn)
10.            printf("BinarySem_Handle二值信号量获取成功!\n\n");
11.            LED1_TOGGLE;
12.     }
13. }
```



5.2 xSemaphoreTakeFromISR()（中断）

xSemaphoreTakeFromISR()是函数 xSemaphoreTake()的中断版本，用于获取信号量，是一个不带阻塞机制获取信号量的函数，获取对象必须由是已经创建的信号量，信号量类型 可以是二值信号量和计数信号量，它与 xSemaphoreTake()函数不同，它不能用于获取互斥量，因为互斥量不可以在中断中使用，并且互斥量特有的优先级继承机制只能在任务中起作用，而在中断中毫无意义。

xSemaphoreTakeFromISR()函数说明

函数原型	xSemaphoreTakeFromISR(SemaphoreHandle_t xSemaphore, signed BaseType_t *pxHigherPriorityTaskWoken)	
功能	在中断中获一个信号量（其实很少在中断中获取信号量）。可以是二值信号量、计数信号量。	
参数	xSemaphore	信号量句柄。
	pxHigherPriorityTaskWoken	一个或者多个任务有可能阻塞在同一个信号量上，调用函数 xSemaphoreTakeFromISR()会唤醒阻塞在该信号量上优先级最高的信号量入队任务，如果被唤醒的任务的优先级大于或者等于被中断的任务的优先级，那么形参 pxHigherPriorityTaskWoken 就会被设置为 pdTRUE，然后在中断退出前执行一次上下文切换，中断退出后则直接返回刚刚被唤醒的高优先级的任务。从 FreeRTOS V7.3.0 版本开始， pxHigherPriorityTaskWoken 是一个可选的参数，可以设置为 NULL。
返回值	获取成功则返回 pdTRUE，没有获取成功则返回 errQUEUE_EMPTY，没有获取成功是因为信号量不可用。 CSDN @不秃也很强	

六、二值信号量同步实验

信号量同步实验是在 FreeRTOS 中创建了两个任务，一个是获取信号量任务，一个是 释放互斥量任务，两个任务独立运行，获取信号量任务是一直在等待信号量，其等待时间是 portMAX_DELAY，等到获取到信号量之后，任务开始执行任务代码，如此反复等待另外任务释放的信号量。释放信号量任务在检测按键是否按下，如果按下则释放信号量，此时释放信号量会唤醒获取任务，获取任务开始运行，然后形成两个任务间的同步，因为如果没按下按键，那么信号量就不会释放，只有当信号量释放的时候，获取信号量的任务才会被唤醒，如此一来就达到任务与任务的同步，同时程序的运行会在串口打印出相关信息。

```
1. /* FreeRTOS头文件 */
2. #include "FreeRTOS.h"
3. #include "task.h"
4. #include "queue.h"
5. #include "semphr.h"
6. /* 开发板硬件bsp头文件 */
7. #include "bsp_led.h"
8. #include "bsp_usart.h"
9. #include "bsp_key.h"
10. /***** 任务句柄 *****/
11. /*
12. * 任务句柄是一个指针，用于指向一个任务，当任务创建好之后，它就具有了一个任务句柄
13. * 以后我们要想操作这个任务都需要通过这个任务句柄，如果是自身的任务操作自己，那么
14. * 这个句柄可以为NULL。
15. */
16. static TaskHandle_t AppTaskCreate_Handle = NULL; /* 创建任务句柄 */
17. static TaskHandle_t Receive_Task_Handle = NULL; /* LED任务句柄 */
18. static TaskHandle_t Send_Task_Handle = NULL; /* KEY任务句柄 */
19.
20. /***** 内核对象句柄 *****/
21. /*
22. * 信号量，消息队列，事件标志组，软件定时器这些都属于内核的对象，要想使用这些内核
23. * 对象，必须先创建，创建成功之后会返回一个相应的句柄。实际上就是一个指针，后续我
24. * 们就可以通过这个句柄操作这些内核对象。
25. *
26. * 内核对象说白了就是一种全局的数据结构，通过这些数据结构我们可以实现任务间的通信，
27. * 任务间的事件同步等各种功能。至于这些功能的实现我们是通过调用这些内核对象的函数
28. * 来完成的
29. *
30. */
31. SemaphoreHandle_t BinarySem_Handle = NULL;
32.
33. /***** 全局变量声明 *****/
34. /*
35. * 当我们在写应用程序的时候，可能需要用到一些全局变量。
36. */
37.
38.
39. /***** 宏定义 *****/
40. /*
41. * 当我们在写应用程序的时候，可能需要用到一些宏定义。
42. */
43.
44.
45. /*
46. *****/
47. * 函数声明
48. *****/
49. /*
50. static void AppTaskCreate(void); /* 用于创建任务 */
51.
52. static void Receive_Task(void* pvParameters); /* Receive_Task任务实现 */
53. static void Send_Task(void* pvParameters); /* Send_Task任务实现 */
54.
55. static void BSP_Init(void); /* 用于初始化板载相关资源 */
56.
57. /*****
58. * @brief 主函数
59. * @param 无
60. * @retval 无
61. * @note 第一步：开发板硬件初始化
62. 第二步：创建APP应用任务
63. 第三步：启动FreeRTOS，开始多任务调度
64. *****/
65. int main(void)
66. {
67. BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
68.
69. /* 开发板硬件初始化 */
70. BSP_Init();
71. printf("这是一个FreeRTOS二值信号量同步实验！\n");
72. printf("按下KEY1或者KEY2进行任务与任务间的同步\n");
73. /* 创建AppTaskCreate任务 */
74. xReturn = xTaskCreate((TaskFunction_t) AppTaskCreate, /* 任务入口函数 */
75. (const char* ) "AppTaskCreate", /* 任务名字 */
76. (uint16_t ) 512, /* 任务栈大小 */
77. (void* ) NULL, /* 任务入口函数参数 */
78. (UBaseType_t ) 1, /* 任务的优先级 */
79. (TaskHandle_t* ) &AppTaskCreate_Handle); /* 任务控制块指针 */
80.
81. /* 启动任务调度 */
82. if (pdPASS == xReturn)
83. vTaskStartScheduler(); /* 启动任务，开启调度 */
84. else
85. return -1;
86.
87. while(1); /* 正常不会执行到这里 */
88. }
89.
90. /*****
91. * @ 函数名 : AppTaskCreate
92. * @ 功能说明: 为了方便管理，所有的任务创建函数都放在这个函数里面
```

```

93.  * @ 参数      : 无
94.  * @ 返回值    : 无
95.  *****/
96. static void AppTaskCreate(void)
97. {
98.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
99.
100.    taskENTER_CRITICAL();          //进入临界区
101.
102.    /* 创建 BinarySem */
103.    BinarySem_Handle = xSemaphoreCreateBinary(); /*xSemaphoreCreateBinary 创建信号量*/
104.    if(NULL != BinarySem_Handle)
105.        printf("BinarySem_Handle二值信号量创建成功!\r\n");
106.
107.    /* 创建Receive_Task任务 */
108.    xReturn = xTaskCreate((TaskFunction_t)Receive_Task, /* 任务入口函数 */
109.                          (const char*) "Receive_Task", /* 任务名字 */
110.                          (uint16_t) 512, /* 任务栈大小 */
111.                          (void*) NULL, /* 任务入口函数参数 */
112.                          (UBaseType_t) 2, /* 任务的优先级 */
113.                          (TaskHandle_t*) &Receive_Task_Handle); /* 任务控制块指针 */
114.    if(pdPASS == xReturn)
115.        printf("创建Receive_Task任务成功!\r\n");
116.
117.    /* 创建Send_Task任务 */
118.    xReturn = xTaskCreate((TaskFunction_t)Send_Task, /* 任务入口函数 */
119.                          (const char*) "Send_Task", /* 任务名字 */
120.                          (uint16_t) 512, /* 任务栈大小 */
121.                          (void*) NULL, /* 任务入口函数参数 */
122.                          (UBaseType_t) 3, /* 任务的优先级 */
123.                          (TaskHandle_t*) &Send_Task_Handle); /* 任务控制块指针 */
124.    if(pdPASS == xReturn)
125.        printf("创建Send_Task任务成功!\n\n");
126.
127.    vTaskDelete(AppTaskCreate_Handle); //删除AppTaskCreate任务
128.
129.    taskEXIT_CRITICAL();          //退出临界区
130. }
131.
132.
133.
134. /*****
135.  * @ 函数名      : Receive_Task
136.  * @ 功能说明    : Receive_Task任务主体
137.  * @ 参数        :
138.  * @ 返回值      : 无
139.  *****/
140. static void Receive_Task(void* parameter)
141. {
142.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
143.     while (1)
144.     {
145.         //获取二值信号量 xSemaphore, 没获取到则一直等待
146.         xReturn = xSemaphoreTake(BinarySem_Handle, /* 二值信号量句柄 xSemaphoreTake获取一个信号量，可以是二值信号量、计数信号量、互斥量。*/
147.                                  portMAX_DELAY); /* 等待时间 */
148.         if(pdTRUE == xReturn)
149.             printf("BinarySem_Handle二值信号量获取成功!\n\n");
150.         LED1_TOGGLE;
151.     }
152. }
153.
154. /*****
155.  * @ 函数名      : Send_Task
156.  * @ 功能说明    : Send_Task任务主体
157.  * @ 参数        :
158.  * @ 返回值      : 无
159.  *****/
160. static void Send_Task(void* parameter)
161. {
162.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
163.     while (1)
164.     {
165.         /* K1 被按下 */
166.         if( Key_Scan(KEY1_GPIO_PORT, KEY1_GPIO_PIN) == KEY_ON )
167.         {
168.             xReturn = xSemaphoreGive( BinarySem_Handle ); /*给出二值信号量 xSemaphoreGive 释放信号量*/
169.             if( xReturn == pdTRUE )
170.                 printf("BinarySem_Handle二值信号量释放成功!\r\n");
171.             else
172.                 printf("BinarySem_Handle二值信号量释放失败!\r\n");
173.         }
174.         /* K2 被按下 */
175.         if( Key_Scan(KEY2_GPIO_PORT, KEY2_GPIO_PIN) == KEY_ON )
176.         {
177.             xReturn = xSemaphoreGive( BinarySem_Handle ); /*给出二值信号量*/
178.             if( xReturn == pdTRUE )
179.                 printf("BinarySem_Handle二值信号量释放成功!\r\n");
180.             else
181.                 printf("BinarySem_Handle二值信号量释放失败!\r\n");
182.         }
183.         vTaskDelay(20);
184.     }
185. }
186. /*****
187.  * @ 函数名      : BSP_Init
188.  * @ 功能说明    : 板级外设初始化，所有板子上的初始化均可放在这个函数里面
189.  * @ 参数        :
190.  * @ 返回值      : 无
191.  *****/
192. static void BSP_Init(void)
193. {
194.     /*
195.      * STM32中断优先级分组为4，即4bit都用来表示抢占优先级，范围为：0~15
196.      * 优先级分组只需要分组一次即可，以后如果有其他的任务需要用到中断，

```

```

197.         * 都统一用这个优先级分组，千万不要再分组，切忌。
198.         */
199.         NVIC_PriorityGroupConfig( NVIC_PriorityGroup_4 );
200.
201.         /* LED 初始化 */
202.         LED_GPIO_Config();
203.
204.         /* 串口初始化 */
205.         USART_Config();
206.
207.         /* 按键初始化 */
208.         Key_GPIO_Config();
209.     }
210. }
211.
212. /*****END OF FILE*****/

```



七、计数信号量实验

计数型信号量实验是模拟停车场工作运行。在创建信号量的时候初始化 5 个可用的信号量，并且创建了两个任务：一个是获取信号量任务，一个是释放信号量任务，两个任务独立运行，获取信号量任务是通过按下 KEY1 按键进行信号量的获取，模拟停车场停车操作，其等待时间是 0，在串口调试助手输出相应信息。释放信号量任务则是信号量的释放，释放信号量任务也是通过按下 KEY2 按键进行信号量的释放，模拟停车场取车操作，在串口调试助手输出相应信息。

```

1.  /* FreeRTOS头文件 */
2.  #include "FreeRTOS.h"
3.  #include "task.h"
4.  #include "queue.h"
5.  #include "semphr.h"
6.  /* 开发板硬件bsp头文件 */
7.  #include "bsp_led.h"
8.  #include "bsp_usart.h"
9.  #include "bsp_key.h"
10. /***** 任务句柄 *****/
11. /*
12.  * 任务句柄是一个指针，用于指向一个任务，当任务创建好之后，它就具有了一个任务句柄
13.  * 以后我们要想操作这个任务都需要通过这个任务句柄，如果是自身的任务操作自己，那么
14.  * 这个句柄可以为NULL。
15.  */
16. static TaskHandle_t AppTaskCreate_Handle = NULL; /* 创建任务句柄 */
17. static TaskHandle_t Take_Task_Handle = NULL; /* Take_Task任务句柄 */
18. static TaskHandle_t Give_Task_Handle = NULL; /* Give_Task任务句柄 */
19.
20. /***** 内核对象句柄 *****/
21. /*
22.  * 信号量，消息队列，事件标志组，软件定时器这些都属于内核的对象，要想使用这些内核
23.  * 对象，必须先创建，创建成功之后会返回一个相应的句柄。实际上就是一个指针，后续我
24.  * 们就可以通过这个句柄操作这些内核对象。
25.  */
26. /* 内核对象说白了就是一种全局的数据结构，通过这些数据结构我们可以实现任务间的通信，
27.  * 任务间的事件同步等各种功能。至于这些功能的实现我们是通过调用这些内核对象的函数
28.  * 来完成的
29.  */
30. /*
31. SemaphoreHandle_t CountSem_Handle = NULL;
32.
33. /***** 全局变量声明 *****/
34. /*
35.  * 当我们在写应用程序的时候，可能需要用到一些全局变量。
36.  */
37.
38.
39. /***** 宏定义 *****/
40. /*
41.  * 当我们在写应用程序的时候，可能需要用到一些宏定义。
42.  */
43.
44.
45. /*
46. *****/
47. * 函数声明
48. *****/
49. */
50. static void AppTaskCreate(void); /* 用于创建任务 */
51.
52. static void Take_Task(void* pvParameters); /* Take_Task任务实现 */
53. static void Give_Task(void* pvParameters); /* Give_Task任务实现 */
54.
55. static void BSP_Init(void); /* 用于初始化板载相关资源 */
56.
57. /*****
58.  * @brief 主函数
59.  * @param 无
60.  * @retval 无
61.  * @note 第一步：开发板硬件初始化
62.         第二步：创建APP应用任务
63.         第三步：启动FreeRTOS，开始多任务调度
64. *****/
65. int main(void)
66. {
67.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
68.
69.     /* 开发板硬件初始化 */
70.     BSP_Init();
71.
72.     printf("这是一个FreeRTOS计数信号量实验！\n");
73.     printf("车位默认为5个，按下KEY1申请车位，按下KEY2释放车位！\n\n");

```

```

74.
75. /* 创建AppTaskCreate任务 */
76. xReturn = xTaskCreate((TaskFunction_t )AppTaskCreate, /* 任务入口函数 */
77.                      (const char* )"AppTaskCreate", /* 任务名字 */
78.                      (uint16_t )512, /* 任务栈大小 */
79.                      (void* )NULL, /* 任务入口函数参数 */
80.                      (UBaseType_t )1, /* 任务的优先级 */
81.                      (TaskHandle_t* )&AppTaskCreate_Handle); /* 任务控制块指针 */
82.
83. /* 启动任务调度 */
84. if(pdPASS == xReturn)
85.     vTaskStartScheduler(); /* 启动任务，开启调度 */
86. else
87.     return -1;
88.
89. while(1); /* 正常不会执行到这里 */
90. }
91.
92. /*****
93. * @ 函数名 : AppTaskCreate
94. * @ 功能说明: 为了方便管理，所有的任务创建函数都放在这个函数里面
95. * @ 参数 : 无
96. * @ 返回值 : 无
97. *****/
98. static void AppTaskCreate(void)
99. {
100.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
101.
102.     taskENTER_CRITICAL(); /* 进入临界区
103.
104.     /* 创建Test_Queue */
105.     CountSem_Handle = xSemaphoreCreateCounting(10, 10);
106.     if(NULL != CountSem_Handle)
107.         printf("CountSem_Handle计数信号量创建成功!\r\n");
108.
109.     /* 创建Take_Task任务 */
110.     xReturn = xTaskCreate((TaskFunction_t )Take_Task, /* 任务入口函数 */
111.                          (const char* )"Take_Task", /* 任务名字 */
112.                          (uint16_t )512, /* 任务栈大小 */
113.                          (void* )NULL, /* 任务入口函数参数 */
114.                          (UBaseType_t )2, /* 任务的优先级 */
115.                          (TaskHandle_t* )&Take_Task_Handle); /* 任务控制块指针 */
116.
117.     if(pdPASS == xReturn)
118.         printf("创建Take_Task任务成功!\r\n");
119.
120.     /* 创建Give_Task任务 */
121.     xReturn = xTaskCreate((TaskFunction_t )Give_Task, /* 任务入口函数 */
122.                          (const char* )"Give_Task", /* 任务名字 */
123.                          (uint16_t )512, /* 任务栈大小 */
124.                          (void* )NULL, /* 任务入口函数参数 */
125.                          (UBaseType_t )3, /* 任务的优先级 */
126.                          (TaskHandle_t* )&Give_Task_Handle); /* 任务控制块指针 */
127.
128.     if(pdPASS == xReturn)
129.         printf("创建Give_Task任务成功!\n\n");
130.
131.     vTaskDelete(AppTaskCreate_Handle); /* 删除AppTaskCreate任务
132.
133.     taskEXIT_CRITICAL(); /* 退出临界区
134. }
135.
136. /*****
137. * @ 函数名 : Take_Task
138. * @ 功能说明: Take_Task任务主体
139. * @ 参数 :
140. * @ 返回值 : 无
141. *****/
142. static void Take_Task(void* parameter)
143. {
144.     BaseType_t xReturn = pdTRUE; /* 定义一个创建信息返回值，默认为pdPASS */
145.     /* 任务都是一个无限循环，不能返回 */
146.     while (1)
147.     {
148.         /* 如果KEY1被单击
149.         if( Key_Scan(KEY1_GPIO_PORT, KEY1_GPIO_PIN) == KEY_ON )
150.         {
151.             /* 获取一个计数信号量 */
152.             xReturn = xSemaphoreTake(CountSem_Handle, /* 计数信号量句柄 */
153.                                     0); /* 等待时间: 0 */
154.             if ( pdTRUE == xReturn )
155.                 printf( "KEY1被按下，成功申请到停车位。 \n" );
156.             else
157.                 printf( "KEY1被按下，不好意思，现在停车场已满! \n" );
158.         }
159.         vTaskDelay(20); /* 每20ms扫描一次
160.     }
161. }
162.
163. /*****
164. * @ 函数名 : Give_Task
165. * @ 功能说明: Give_Task任务主体
166. * @ 参数 :
167. * @ 返回值 : 无
168. *****/
169. static void Give_Task(void* parameter)
170. {
171.     BaseType_t xReturn = pdTRUE; /* 定义一个创建信息返回值，默认为pdPASS */
172.     /* 任务都是一个无限循环，不能返回 */
173.     while (1)
174.     {
175.         /* 如果KEY2被单击
176.         if( Key_Scan(KEY2_GPIO_PORT, KEY2_GPIO_PIN) == KEY_ON )
177.         {

```

```

178.             /* 获取一个计数信号量 */
179.     xReturn = xSemaphoreGive(CountSem_Handle); //给出计数信号量
180.     if ( pdTRUE == xReturn )
181.         printf( "KEY2被按下，释放1个停车位。\\n" );
182.     else
183.         printf( "KEY2被按下，但已无车位可以释放! \\n" );
184.     }
185.     vTaskDelay(20);      //每20ms扫描一次
186. }
187. }
188. /*****
189.  * @ 函数名   :   BSP_Init
190.  * @ 功能说明 :   板级外设初始化，所有板子上的初始化均可放在这个函数里面
191.  * @ 参数     :
192.  * @ 返回值   :   无
193.  *****/
194. static void BSP_Init(void)
195. {
196.     /*
197.     * STM32中断优先级分组为4，即4bit都用来表示抢占优先级，范围为：0~15
198.     * 优先级分组只需要分组一次即可，以后如果有其他的任务需要用到中断，
199.     * 都统一用这个优先级分组，千万不要再分组，切忌。
200.     */
201.     NVIC_PriorityGroupConfig( NVIC_PriorityGroup_4 );
202.
203.     /* LED 初始化 */
204.     LED_GPIO_Config();
205.
206.     /* 按键初始化 */
207.     Key_GPIO_Config();
208.
209.     /* 串口初始化 */
210.     USART_Config();
211.
212.
213. }
214. }
215.
216. /*****END OF FILE*****/

```



八、实验现象

串口配置

端口

COM13 US1

波特率

115200

数据位

8

停止位

1

校验位

无

操作

关闭串口

这是一个[野火]-STM32全系列开发板-FreeRTOS二值信号量同步实验!

按下KEY1或者KEY2进行任务与任务间的同步

BinarySem_Handle二值信号量创建成功!

创建Receive_Task任务成功!

创建Send_Task任务成功!

BinarySem_Handle二值信号量释放成功!

BinarySem_Handle二值信号量获取成功!

BinarySem_Handle二值信号量释放成功!

BinarySem_Handle二值信号量获取成功!

BinarySem_Handle二值信号量释放成功!

BinarySem_Handle二值信号量获取成功!

CSDN @不秃也很强

串口配置

端口

COM13 US1

波特率

115200

数据位

8

停止位

1

校验位

无

操作

关闭串口

这是一个[野火]-STM32全系列开发板-FreeRTOS计数信号量实验!

车位默认值为5个，按下KEY1申请车位，按下KEY2释放车位!

CountSem_Handle计数信号量创建成功!

创建Take_Task任务成功!

创建Give_Task任务成功!

KEY1被按下，成功申请到停车位。

KEY1被按下，成功申请到停车位。

KEY1被按下，成功申请到停车位。

KEY1被按下，成功申请到停车位。

KEY2被按下，释放1个停车位。

KEY2被按下，释放1个停车位。

KEY2被按下，释放1个停车位。

KEY2被按下，释放1个停车位。

KEY2被按下，释放1个停车位。

KEY2被按下，但已无车位可以释放!

KEY1被按下，成功申请到停车位。

KEY2被按下，释放1个停车位。

CSDN @不秃也很强